Firmware sizing prototype

Purpose: answer "does everything fit in the MCU?" with a real compile instead of an estimate.

This is not the product firmware and it has **not been run on hardware**. The register-level code (scan ISR, ADC touch, flash writes) follows the vendors' reference headers and HAL sources but is untested.

It's a bare-metal C port of everything the tree has to run. The shared code is the same for every MCU:

File	What
`shows.c`	the 15 light shows, integer-only
`face.c`	Pine's face parts (bitmaps) and 18 behaviours
`scene.c`	scenes, face overlays, face morph and the 6 transitions
`brain.c`	emotions (Q16) and the behaviour state machine
`gen_tables.py` → `tables.c`, `geom.h`	per-LED constants from `hardware/led_map.json` (no float on the MCU), incl. which LEDs sit on which anode phase
`pp.h`, `libc.c`	shared declarations, `memset` / `memcpy`
`build.py`	builds each target, then reports flash, RAM, call-graph stack depth, soft-float check and the fit

The drivers are per MCU, in `py32f003/` (the product) and `ch32v003/` (the original choice, kept for the comparison):

File	PY32F003F18P6	CH32V003F4P6
`main.c`	vector table, HSI 24 MHz (no PLL), 50 fps SysTick loop, xorshift RNG	startup, 48 MHz PLL, 50 fps SysTick loop
`scan.c`	TIM1 charlieplex BCM scan ISR, bits 0+1 in one interrupt, 256-entry pin → `MODER` table	TIM2 scan ISR, one interrupt per bit
`touch.c`	ADC charge-sharing touch on PA7 (precharge from Vrefint), gestures, LED ambient-light sense	same on PD4
`store.c`	Pine's bond in a ring of 8 × 128-byte flash pages; page program from a RAM routine, as in Puya's HAL	16 slots in the last 64-byte page
`*.ld`	63 KB flash + 1 KB bond store, 8 KB RAM	16 KB − 64 B flash, 2 KB RAM

Build

```
python firmware/sizetest/build.py
```

It builds every target whose compiler it finds and skips the others:

- PY32F003: Arm GNU Toolchain (`arm-none-eabi-gcc`, tested with 14.2.Rel1 in `C:\Program Files (x86)\Arm GNU Toolchain arm-none-eabi\`)
- CH32V003: xPack RISC-V GCC (`riscv-none-elf-gcc`, tested with 15.2.0 in `%LOCALAPPDATA%\xpack\`)

Pass `py32f003` or `ch32v003` to build just one. Output goes to `build/<target>/`.

Result (2026-09-24)

	PY32F003F18P6 (product)	CH32V003F4P6 (original)
Flash, LTO	**17 204 B of 64 512 B (27 %)**	20 856 B of 16 320 B (128 %): doesn't fit
RAM: static + worst-case stack	**1 892 B of 8 192 B (23 %)**	1 776 B of 2 048 B (87 %)
Soft-float code	none	none
Compiler warnings (`-Wall -Wextra`)	0	0

The CH32V003 overflowed, so the product MCU was changed to the PY32F003F18P6. The shared code is 30–40 % smaller as Cortex-M0+ Thumb code (single-instruction multiply instead of `__mulsi3` calls). The full breakdown, the CPU-time estimate at 24 MHz and the options that were considered are in docs/animations.md.